

Documentação — Trabalho DCGAN: Geração de Rostos (CelebA)

1. Resumo

Este trabalho implementa uma **DCGAN** (*Deep Convolutional Generative Adversarial Network*) para gerar imagens de rostos a partir do dataset **CelebA**. O projeto parte de uma base didática de DCGAN sobre o MNIST e a evolui para um problema mais difícil (rostos coloridos em 64×64), tendo como eixo central a **curadoria de dados**: em vez de treinar com todas as imagens, identifica-se e separa-se as imagens fora do padrão, treinando a rede apenas num conjunto homogêneo. O resultado é acompanhado por um **experimento controlado** que demonstra, isoladamente, o efeito da curadoria sobre a qualidade das imagens geradas.

O notebook foi desenvolvido e executado no **Kaggle** (GPU), de forma autossuficiente (download do dataset, curadoria, treino e avaliação).

2. Objetivo

Treinar uma rede generativa capaz de, a partir de **vetores de ruído aleatório** (pontos do espaço latente), produzir rostos coerentes — ou seja, fazer a rede sair de pontos aleatórios e chegar a uma imagem plausível. O foco metodológico não está apenas no modelo, mas em **tratar o dataset antes do treino**, demonstrando que a qualidade dos dados é determinante para o resultado de uma GAN.

3. Como funciona uma DCGAN (visão geral)

Uma GAN treina duas redes que competem entre si:

- O **Gerador (G)** recebe um vetor de ruído **z** e tenta transformá-lo numa imagem realista.
- O **Discriminador (D)** recebe imagens (reais do dataset ou falsas do gerador) e tenta distinguir umas das outras.

O treino é adversarial: o gerador melhora tentando enganar o discriminador, e o discriminador melhora tentando não ser enganado. É importante notar que a rede opera sobre **matrizes de pixels** — ela não entende "contexto" semântico, apenas aprende a estatística espacial das imagens. Por isso, quanto mais **homogêneo** o conjunto de treino, mais fácil é a convergência.

4. Conjunto de dados e curadoria

4.1. Dataset

Foi utilizado o **CelebA** ([jessicali9530/celeba-dataset](https://github.com/jessicali9530/celeba-dataset)), composto por **202.599** imagens de rostos de celebridades já alinhados ([img_align_celeba](#)). O dataset acompanha dois arquivos de anotação aproveitados na curadoria: [list_attr_celeba.csv](#) (40 atributos binários por imagem) e [list_landmarks_align_celeba.csv](#) (coordenadas de olhos, nariz e boca).

4.2. Identificação do problema do dataset

Embora o CelebA venha alinhado, ele ainda contém grande **variabilidade**: óculos, chapéus, barbas, rostos de perfil ou inclinados, e composições fora do padrão. Para uma GAN nessa resolução, essa variabilidade dispersa a capacidade do gerador, levando a rostos borrados, assimétricos ou "derretidos". O problema identificado, portanto, foi a **heterogeneidade de forma e acessórios** entre as instâncias.

4.3. Curadoria aplicada

A curadoria é feita programaticamente na classe [CelebACuradoDataset](#), combinando dois filtros que usam as próprias anotações do dataset para **separar as imagens dentro e fora do padrão**:

1. **Filtro de atributos** — mantém apenas imagens **sem óculos, sem chapéu e sem barba** (`Eyeglasses == -1, Wearing_Hat == -1, No_Beard == 1`). Remove acessórios que quebram a homogeneidade de forma.
2. **Filtro geométrico (frontalidade)** — usa as coordenadas dos olhos e do nariz para medir a **simetria** do rosto. Calcula a distância horizontal nariz–olho esquerdo e nariz–olho direito e exige que a razão entre elas fique entre **0,75 e 1,333** (ambas positivas). Isso mantém apenas rostos aproximadamente **frontais** (sujeito centralizado).

Etapas	Quantidade de imagens
Imagens originais	202.599
Após a curadoria	63.183
Proporção mantida	~31,2%

Ou seja, cerca de **69% das imagens foram descartadas** por estarem fora do padrão, e o treino ocorreu sobre as ~63 mil imagens mais consistentes em **proporção, forma e matiz**.

4.4. Pré-processamento

Cada imagem mantida passa pelas seguintes transformações:

- `CenterCrop(160)` — recorta a região central, reforçando a centralização do rosto.
- `Resize(64, 64)` com interpolação **LANCZOS** — reduz para 64×64 preservando detalhes.
- `RandomHorizontalFlip(0.5)` — espelhamento aleatório; como um rosto espelhado continua plausível, isso aumenta a variedade de poses sem custo de coleta.
- `Normalize([0.5]×3, [0.5]×3)` — mapeia os pixels para o intervalo **[-1, 1]**, casando com a função `Tanh` da saída do gerador.

5. Arquitetura das redes

Ambas as redes são totalmente convolucionais, no padrão DCGAN, operando em RGB 64×64.

5.1. Gerador (3.576.704 parâmetros)

Projeta o vetor latente **z** (dimensão 100) e o expande por camadas de **convolução transposta**, dobrando a resolução a cada etapa: $1 \times 1 \rightarrow 4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64$. Entre as camadas usa `BatchNorm + ReLU`, e `Tanh` na saída (gerando valores em [-1, 1]).

5.2. Discriminador (2.765.568 parâmetros)

Reduz a imagem 64×64 até um escalar por camadas `Conv2d` com `LeakyReLU(0.2)`. Cada convolução é envolvida por **Spectral Normalization**, e a saída é um **logit** (sem `Sigmoid`), pois a função de perda utilizada é a `BCEWithLogitsLoss`.

6. Técnicas de treinamento e justificativas

Técnica	Função no projeto	Por que foi usada
<code>BCEWithLogitsLoss</code>	Perda adversarial	Mais estável numericamente que <code>Sigmoid + BCELoss</code>
Spectral Normalization (no D)	Limita o "poder" de cada camada do discriminador	Evita que o D domine e o treino divirja — principal causa de instabilidade em GANs

Técnica	Função no projeto	Por que foi usada
TTUR ($lr_G = 2e-4$, $lr_D = 1e-4$)	Taxas de aprendizado diferentes	D converge mais rápido; LR menor para ele equilibra o jogo
AMP (precisão mista)	Treino em parte com float16	Acelera o treino e reduz uso de memória na GPU
Gradient clipping ($max_norm=1.0$, no G)	Limita a norma do gradiente	Estabiliza o gerador, evitando passos abruptos
Label smoothing ($REAL_LABEL = 0.9$)	Rótulo "real" suavizado	Impede o discriminador de ficar confiante demais
Inicialização $Normal(0, 0.02)$	Pesos iniciais	Recomendação do artigo original do DCGAN
Otimizador Adam $betas=(0.5, 0.999)$	Otimização	$\beta_1 = 0.5$ reduz o momento, recomendado para GANs

Hiperparâmetros principais: $latent_dim = 100$, $img_size = 64$, $batch_size = 512$, $epochs = 150$. Com 63.183 imagens e batch 512, são **123 batches por época** (≈ 18.450 iterações no total).

7. Resultados

7.1. Modelo final (150 épocas, conjunto curado)

O treino foi estável do início ao fim, sem divergência nem colapso de modo. Na última época, as perdas médias foram $Loss_D \approx 0,77$ e $Loss_G \approx 2,03$ — um regime equilibrado, em que o discriminador não "vence" (não vai a zero) e o gerador continua aprendendo. As imagens finais mostram rostos **nítidos, frontais e variados**, com enquadramento consistente.

Arquivos gerados (em `/kaggle/working/samples/`): `real_samples.png` (amostra do dataset curado), `training_losses.png` (curvas de perda), `final_generated_faces.png` (rostos gerados) e os checkpoints em `/kaggle/working/checkpoints/`.

7.2. Experimento controlado: com × sem curadoria

Para isolar o efeito da curadoria, foram treinadas **duas redes idênticas** (mesma arquitetura, mesma inicialização via semente fixa, mesmas 20 épocas e o **mesmo número de imagens**,

30.000 em cada). A única variável foi o conjunto de treino: um curado (frontais, sem acessórios) e outro aleatório sem filtro. O mesmo vetor de ruído foi usado nos dois para tornar os resultados diretamente comparáveis.

O resultado mostra o lado **curado** com rostos mais frontais, simétricos e estruturados, enquanto o lado **sem curadoria** apresenta mais poses irregulares, acessórios, fundos variados e artefatos. Como todas as demais variáveis foram controladas, a diferença é atribuível **exclusivamente à curadoria** — uma evidência experimental, não uma impressão.

8. Critérios atendidos (alinhamento com o que o professor pediu)

Critério enfatizado pelo professor	Como o trabalho atende
Curadoria dos dados como peça central	Pipeline de curadoria ativo (CelebACuradoDataset) que filtra o dataset antes do treino
Identificar o problema do dataset	Diagnóstico explícito da heterogeneidade (acessórios, rostos não frontais) e seu impacto na GAN
Não adianta "espremer" tudo na IA	Em vez de jogar as 202 mil imagens na rede, treina-se nas 63 mil curadas; mais modelo não substitui dados melhores
Usar a IA / algoritmos para separar as imagens	Uso programático dos CSVs de atributos e landmarks para classificar e separar dentro/fora do padrão
Retornar apenas imagens "concêntricas" (centralizadas)	CenterCrop(160) + filtro de simetria que mantém só rostos frontais e centralizados
Mesma relação de proporção, forma e matiz	Conjunto final homogêneo: rostos frontais, sem acessórios, mesmo enquadramento e escala
Falta de variabilidade no formato facilita	Demonstrado pelo experimento controlado "com × sem curadoria"
A GAN trabalha com matriciamento, não contexto	Justificativa adotada para priorizar homogeneidade visual em vez de quantidade

Critério enfatizado pelo professor	Como o trabalho atende
Não há número mínimo de imagens; depende do problema	Optou-se por 63 mil imagens curadas em vez das 202 mil brutas — qualidade sobre quantidade
Sair de pontos aleatórios até chegar em algo	O gerador parte de ruído z no espaço latente e o transforma em rostos coerentes

9. Limitações e melhorias possíveis

- **Resolução de 64×64:** limita o detalhe fino (textura de pele, fios de cabelo). Resoluções maiores (128/256) exigiriam arquiteturas progressivas e mais tempo de treino.
- **Curadoria mais fina:** poderiam ser adicionados filtros de iluminação ou de qualidade de imagem; ou um detector de rosto dedicado para recorte mais preciso.
- **Métricas quantitativas:** a avaliação foi visual e pelas curvas de perda; métricas como FID dariam uma medida objetiva de qualidade/diversidade.
- **Persistência no Kaggle:** como os resultados ficam em [/kaggle/working](#), é necessário baixar os arquivos (ou usar *Save Version*) antes de encerrar a sessão.

10. Como reproduzir

1. Abrir o notebook no **Kaggle** com **GPU** ativada.
2. Executar as células na ordem. O dataset é baixado automaticamente via [kagglehub](#).
3. A curadoria roda antes do treino e imprime as estatísticas (202.599 → 63.183).
4. O treino (150 épocas) salva amostras a cada 5 épocas e checkpoints a cada 10, em [/kaggle/working/](#).
5. As células finais geram as curvas de perda, o grid de rostos finais e a comparação "com × sem curadoria".
6. Baixar os PNGs e o checkpoint de [/kaggle/working/](#) ao final.